

CLASSIFICAÇÃO DE INFECÇÕES ATRAVÉS DE RADIOGRAFIAS DO TÓRAX

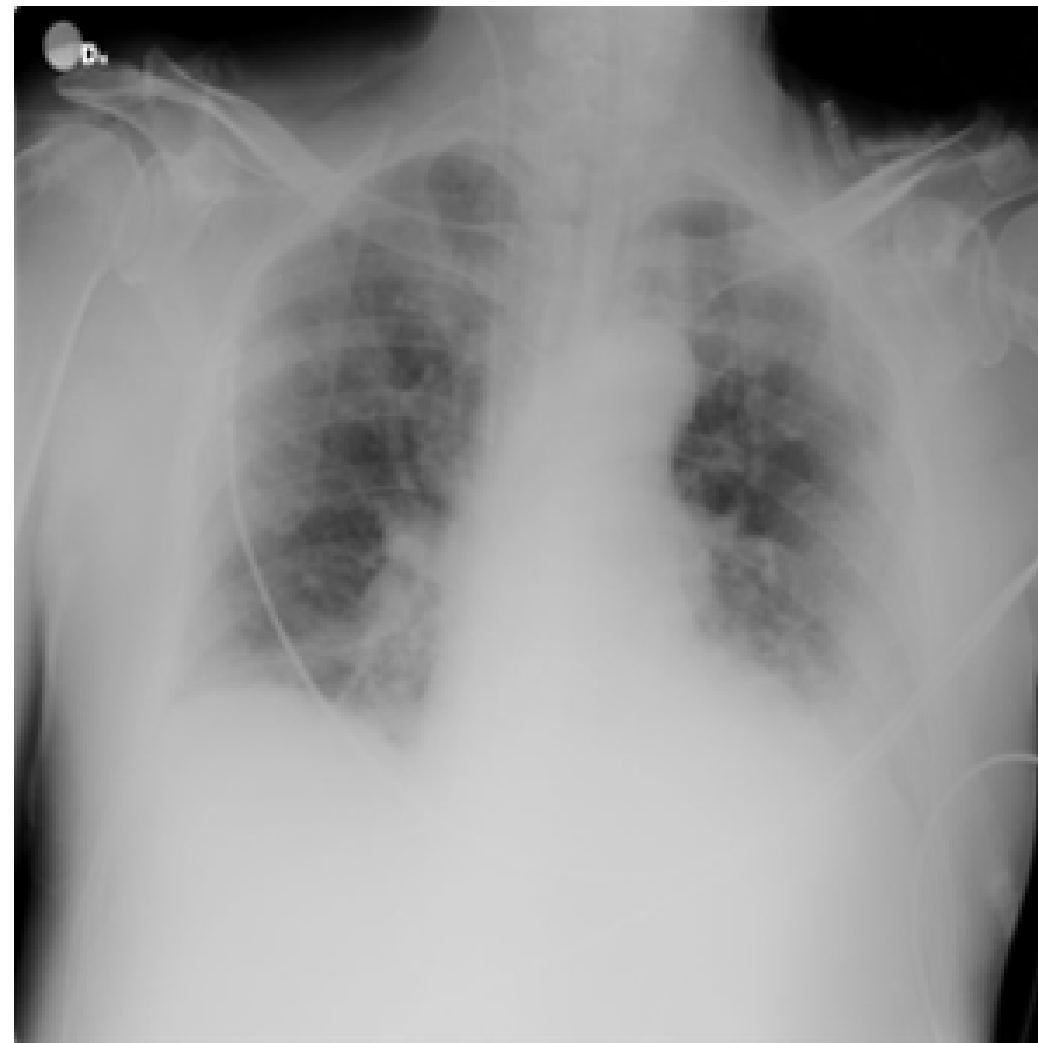
- GUSTAVO FURINI
- JULIANO PROENÇA
- LEONARDO NERVINO
- LUCCA LIBANORI
- RENAN BELEM
- VITORIA IZABEL

INTODRUÇÃO

- Na pandemia as radiografias do tórax foram crucias para o tratamento e a contenção;
- Ajudam no diagnóstico de infecções mas análise é manual e requer tempo;
- Objetivo: Testar diversas técnicas de Visão Computacional para auxiliar na detecção de infecções.

PROTOCOLO EXPERIMENTAL

- A base de dados utilizada contém 21.165 imagens totais, classificadas em quatro categorias: 3.616 imagens de COVID-19, 1.345 de infecções não relacionadas à COVID-19 (como pneumonia viral ou bacteriana), 10.192 normais e 6.012 com opacidade pulmonar (Lung Opacity).
- Foi disponibilizada pela Universidade do Catar



METODOLOGIA

- Criamos as pastas de treino e teste;
- Dividimos as imagens utilizando o `train_test_split`;
- 70% para treino e 30% para teste;
- Redimensionamos para com o `input_shape = (128,128,3)`
- Movemos as imagens para as respectivas pastas;
- Realizamos os três experimentos.

EXPERIMENTO I

- Abordagem com os dados como eles estão;
- Não utilizamos nenhuma técnica adicional como Data Augmentation ou Transfer Learning;
- O modelo apresenta 5 camadas convolucionais;
- Com um Dropout para evitar o overfitting;
- Ativações relu para diminuir a esparsidade;
- E a camada final com softmax para prever as probabilidades das classes.

```
model = models.Sequential()

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape_))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=input_shape_))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(128, (3, 3), activation='relu', input_shape=input_shape_))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=input_shape_))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape_))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Dropout(0.25))
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))
```

EXPERIMENTO II

- Utilizamos o mesmo modelo do experimento anterior, mas com Data Augmentation na base;
- Especificamos as classes com menos imagens para o balanceamento
- Aumentamos 1000 imagens para cada uma dessas classes
- Criamos uma variável com contador para quando atingisse o limite parava o loop

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal", input_shape=(input_shape[0], input_shape[1], 3)),
    tf.keras.layers.RandomRotation(0.15),
    tf.keras.layers.RandomZoom(0.3),
    tf.keras.layers.RandomBrightness(0.1),
    tf.keras.layers.RandomContrast(0.1),
])

target_classes = [0, 3]
augmentation_target_count = 1000

# Contadores para rastrear o número de imagens aumentadas
augmented_count = {0: 0, 3: 0}

# Listas para armazenar as imagens e labels aumentadas
augmented_images = []
augmented_labels = []

# Loop para aplicar data augmentation nas classes
for images, labels in train_ds:
    for i in range(len(labels)):
        label = labels[i].numpy()
        # Continua aplicando data augmentation até atingir o número desejado para cada classe
        while label in target_classes and augmented_count[label] < augmentation_target_count:
            # Aplica data augmentation
            augmented_image = data_augmentation(tf.expand_dims(images[i], 0))
            augmented_images.append(augmented_image[0])
            augmented_labels.append(label)
            augmented_count[label] += 1

        # Se já tiver atingido 1000 imagens para ambas as classes, saia do loop
        if all(count >= augmentation_target_count for count in augmented_count.values()):
            break
    if all(count >= augmentation_target_count for count in augmented_count.values()):
        break
if all(count >= augmentation_target_count for count in augmented_count.values()):
    break
```

EXPERIMENTO III

- Utilizamos a VGG16 já pré-treinada com imagens do ImageNet e aplicamos a Transfer Learning
- Carregamos o modelo com os pesos do ImageNet sem as camadas de classificação superiores.
- Congelamos as camadas, preservando os pesos pré-treinados.

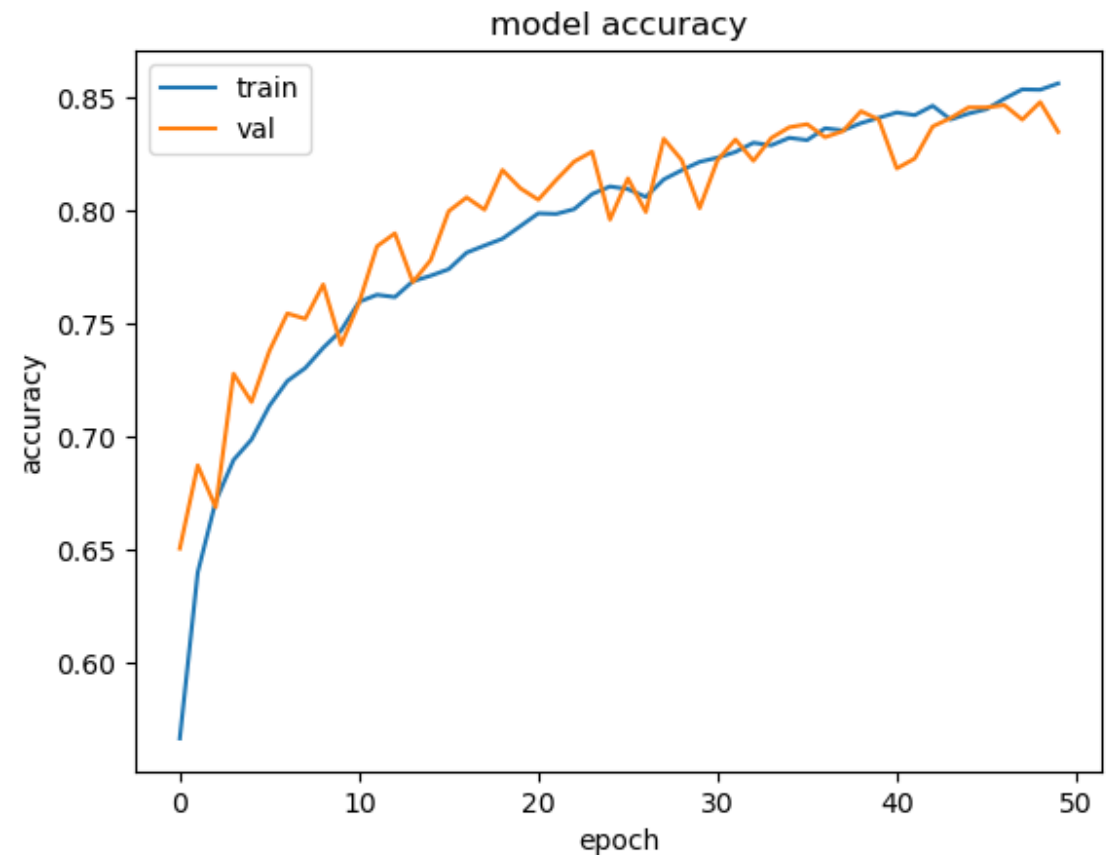
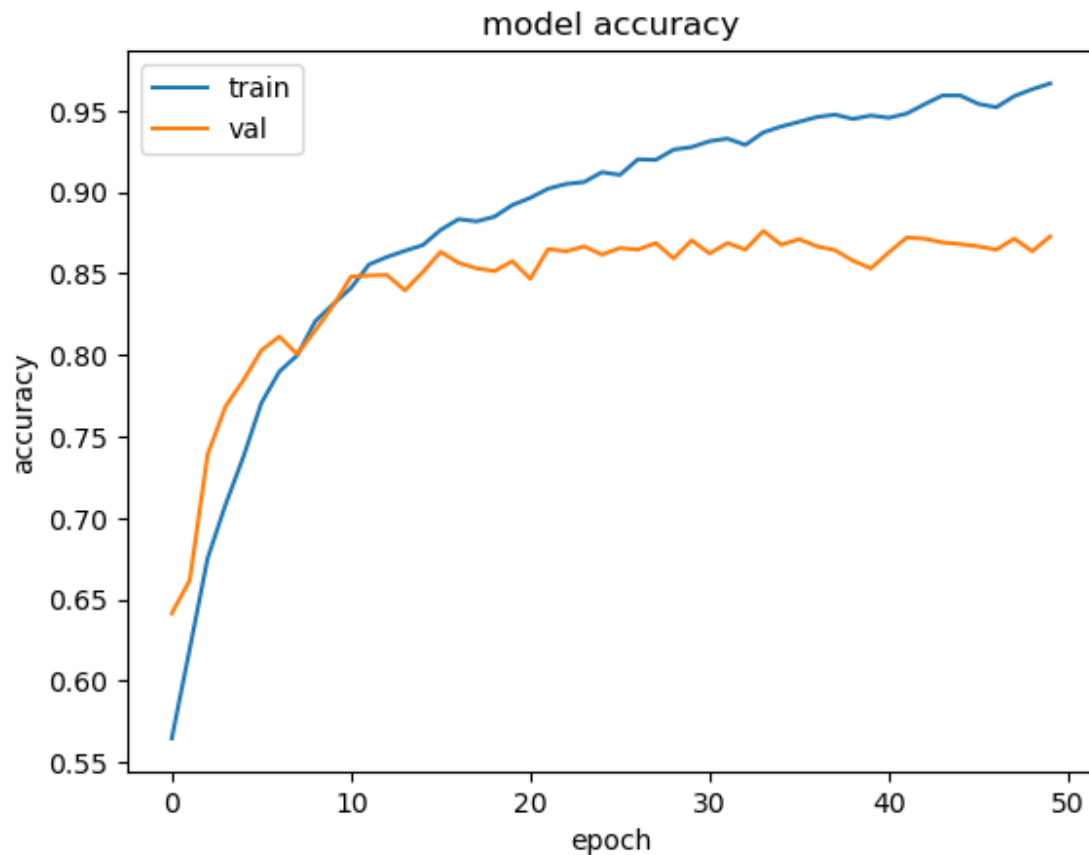
```
vgg_conv = VGG16(weights='imagenet', include_top=False, input_shape=(128, 128, 3))

vgg_conv.trainable = False

model = Sequential()
model.add(vgg_conv)
model.add(Flatten()) #GlobalAvgPooling()
model.add(Dense(2048, activation='relu'))
model.add(Dense(1024, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(num_classes, activation='softmax'))
```

RESULTADOS

- Comparação do treino entre o Experimento I (com Data Augmentation) e Experimento II (sem Data Augmentation)



RESULTADOS

Modelos	Acurácia	F1-Score	Recall	Precisão
Experimento I	88%	88%	87%	90%
Experimento II	87%	87%	89%	85%
Experimento III	90%	91%	91%	90%

CONCLUSÃO

- Os três modelos foram promissores;
- O mesmo modelo com e sem Data Augmentation tiveram desempenho parecidos, o que pode ser que as técnicas de aumento de dados não foram desafiadoras ou a quantidade de imagens aumentadas não foram o suficiente
- O modelo com transferência de aprendizado teve a melhor acurácia, o que mostra que foi bom o desempenho ao ser pré-treinado em imagens genéricas.



OBRIGADO.